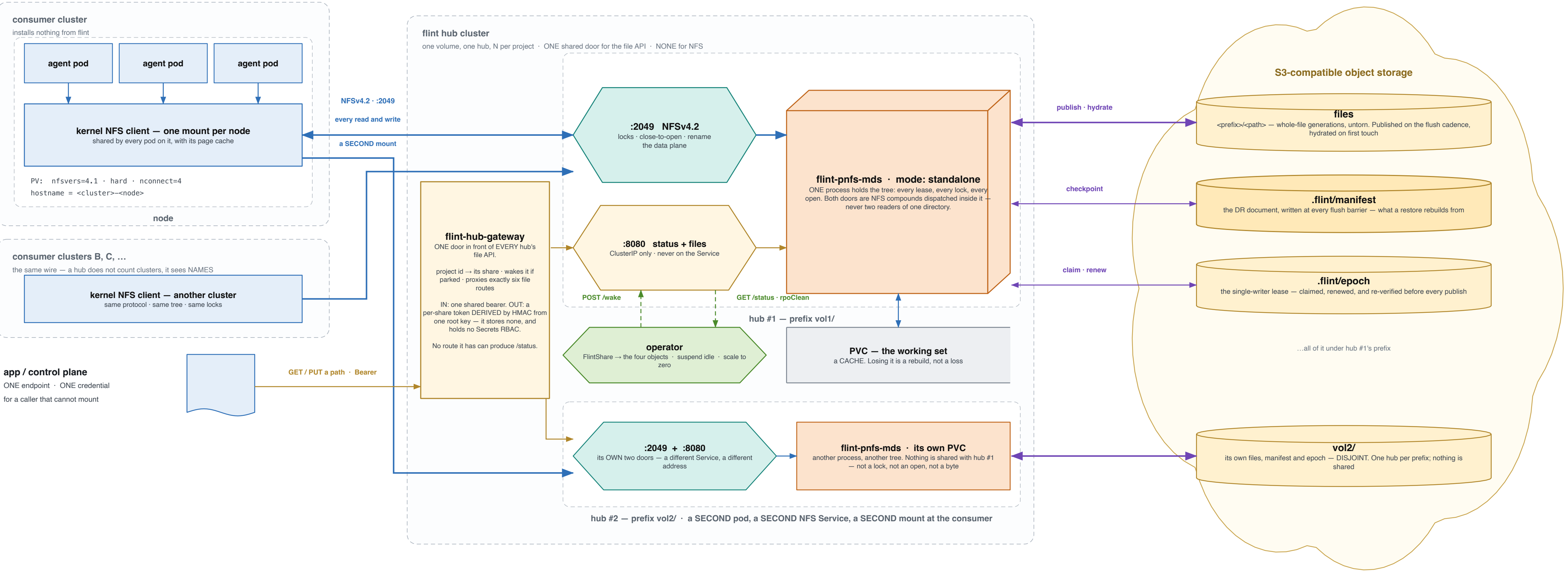


Data flow

One process holds the tree, so N pods in N clusters see one filesystem. The file API can be fronted by one door; the mount cannot.



THE TWO DOORS FRONT DIFFERENTLY, and that is lite's real asymmetry. The FILE API can be multiplexed: flint-hub-gateway resolves a project id to its share, wakes it if it is parked, and proxies six file routes — so a fleet reaches three thousand shares through ONE endpoint and ONE credential, exactly as forge's door fronts every repository. THE MOUNT CANNOT. NFS needs a routable address per hub (`spec.service.advertiseAddress → status.address`), so a second volume is a second pod, a second Service and a second mount at the consumer. Nothing multiplexes NFS.

AND THE GATEWAY EXISTS BECAUSE PER-SHARE EXPOSURE DOES NOT SCALE. A hub's file API answers at a HEADLESS in-cluster Service the operator deliberately never makes routable: 3000 ClusterIPs exhaust a GKE-default /20 at about 2048 shares — and once the allocator is dry, a new share's CONSUMER Service cannot be created either, so one tenant's share count would stop every other tenant mounting anything — while NetworkPolicy cannot be the guard (off by default, hubNamespaces defaults to [], some CNIs ignore every rule in silence). One gateway can be guarded; three thousand cannot.

ONE KEY, NOT THREE THOUSAND SECRETS. Inbound is one shared bearer, re-read every 10 s so rotation is a Secret edit with no restart. Outbound is per-share and DERIVED — HMAC-SHA256(root, endpoint : bucket : keyPrefix : version) — so the gateway stores no hub credential and has no Secrets RBAC at all; the alternative was `get secrets` in every tenant namespace, where those tenants' S3 keys live. The caller's own credential is never forwarded upstream, and no route the gateway has can produce /status.

ONE PROCESS IS THE COHERENCE AUTHORITY, and that is why each hub is one pod. Every lease, lock and open lives in it, so N pods across N clusters see one tree — held to one by Recreate, an exclusive flock and the RWO attach fence inside the cluster, and by the epoch cell in the bucket across clusters. A second hub on ONE prefix is split-brain: it loses the CAS and is fenced. Two hubs on two prefixes, as drawn, is the ordinary configuration.

A HUB DOES NOT COUNT CLUSTERS — IT SEES CLIENT NAMES. The NFSv4 client identity is the hostname and nothing else, so two clusters sharing a node name are ONE client, and RFC 8881 §18.35.5 requires the server to read the second as the first rebooting: the incumbent's locks and opens go, silently, correctly, on a false premise. Set `hostname = <cluster>-<node>`, or `nfs.nfs4_unique_id`.

THE DISK IS A CACHE AND THE BUCKET IS THE COPY, so the recovery point is the flush cadence. Cold files evict at a disk watermark and hydrate on first touch, and both doors SAY SO rather than hanging: NFS parks the caller with `NFS4ERR_DELAY`, HTTP answers 503 + Retry-After. A file request on a parked share WAKES it — measured at 11 s on kind — which is why the gateway watches the CR and never polls a hub: a poll counts as activity and would pin every share it touched awake.

REACHABILITY IS THE BOUNDARY ON :2049 — it authenticates nobody, and kube-proxy SNATs a remote client to an address in the hub's own cluster before the packet arrives (1486 of 1486 connections on a three-cluster rig), so networkPolicy cannot draw it: use peering, security groups or a gateway. And nothing here is a complete oracle of ACTIVITY: the gateway sees every file-API call, but consumers dial each hub's :2049 directly, so a partitioned cluster's busy agents look exactly like idle ones and POST /wake has to be driven by whatever actually knows work is happening.

— data plane — every read and write, over the wire — durable path — each hub is the only writer of ITS prefix — control plane — never reads a file — the file API — ONE shared door in front of every hub

What the abbreviations mean

NFSv4.2 one protocol, one long-lived TCP flow, and a client already in every node's kernel	close-to-open NFS's coherence rule: what a writer closed, the next opener sees	AUTH_SYS the flavour port 2049 speaks: the client asserts its own uid, and nothing checks it	nconnect how many TCP connections one mount opens. Below 2 there is no trunking to spread
NFS4ERR_DELAY the server telling a caller to come back — how a cold read parks instead of hanging	epoch cell the object that says who the hub is. A second hub loses the CAS and is fenced	flush cadence how often closed generations publish. It is the recovery point, stated as a number	hydrate fetching an evicted file back from the bucket on first touch, into its own inode
evict · watermark cold files leave the PVC at a disk watermark — a marker is written before the truncate	.flint/manifest the DR document: the metadata checkpoint a restore rebuilds the tree from	rpoClean /status answering "can the bucket rebuild this volume right now?" — ask it before deleting a PVC	FlintShare the custom resource one lite volume is declared as. One volume, one hub, N of them per project
flint-hub-gateway the proxy that fronts every hub's file API: one endpoint, one inbound credential	headless Service a Service with no ClusterIP — addressable inside the cluster, routable from nowhere	derived token a per-share credential computed from one root key by HMAC, so nothing has to store or fan out 3000 of them	project id · volume id the labels a share is addressed by through the gateway, instead of by its Service name
Recreate the Deployment strategy that stops the old pod before it starts the new one	RWO attach fence the CSI attachment that makes two hub pods on one PVC impossible, not merely unlikely	nfs.nfs4_unique_id the client-side kernel parameter that gives a node a client identity a rename survives	capture · generation a mutation-complete record of what changed, and the closed unit a flush publishes